Diagrama de clases juego

domingo, 25 de octubre de 2026 11:58 a.m.

Juego
- short nivelActual
- short dificultad
- bool pausado
- bool terminado
- Personaje* Kapax
- Nivel* nivel
+ Juego (short dificultad)
+ void iniciar
+ void cargarNivel (short n)
+ void actualizar (float dt)
+ void siguienteNivel ( )
+ void reiniciarNivel ( )
+ void pausar ( )
+ bool juegoTerminado ( )
+ ~Juego ( )
Entidad
# float x
# float y
# float Vx
# float Vy
# float ancho
# float alto
# bool activo
+ Entidad (float x, float y, float ancho, float alto)
+ virtual void actualizar (float dt)
+ void mover (float dx, float dy)
+ bool colisionaCon (const Entidad& otra)
+ void desactivar ( )
Personaje
- short vidas
- short salud
- bool enSuelo
- short saltosRestantes
- bool dobleSaltoActivo
- bool invulnerable
- float tiempoInvulnerable
- bool aturdido
- float tiempoAturdido
- float factorVelocidad
- Gift* giftAlmacenado
+ Personaje (float x, float y)
+ void actualizar (float dt)
+ void moverHorizontal (short dir)
+ void moverVertical (short dir)
+ void saltar ( )
+ void aplicarGravedad (float dt)
+ void recibirGolpe (short danio)
+ void recogerGift (Gift* g)
+ void consumirGift ( )
+ void aturdir (float seg)
+ void activarInvulnerabilidad (float seg)
+ void habilitarDobleSalto ( )
+ void perderVida ( )
+ void reaparecer (float x, float y)
+ bool estaVivo ( )
+ short estadoActual ( )
Nivel
# short numero
# float tiempoMeta
# float tiempoTranscurrido
# bool completado
# bool fallido
# Personaje* jugador
# Dificultad* dificultad
# vector <Enemigo*> enemigos
# vector <Trampa*> trampas
# vector <Gift*> gifts
+ Nivel (Personaje* p, short dificultad)
+ virtual void cargar ( )
+ virtual void actualizar (float dt)
+ virtual bool verificarVictoria ( )
+ void actualizarTemporizador (float dt)
+ void resolverColisiones ( )
+ bool verificarDerrota ( )
+ void limpiarInactivos ( )
+ ~Nivel ( )
Gift
# bool recogido
# float velocidadCaida
+ Gift (float x, float y)
+ virtual void aplicarEfecto (Personaje& p)
+ void actualizar (float dt)
NivelRio
- float velocidadCorriente
- float incrementoCorriente
- float scrollY
- float anchoRio
- float intervaloGeneracion
- float tiempoDesdeGeneracion
+ void cargar ( )
+ void actualizar (float dt)
+ bool verificarVictoria ( )
+ void aplicarCorriente (float dt)
+ void incrementarViolencia (float dt)
+ void generarObstaculos ( )
Dificultad
- short nivel
- float multiplicadorCorriente
- float cadenciaLanzamiento
- float velocidadProyectil
- float densidadTrampas
- float probabilidadGift
- float epsilonAgente
- float alfaAgente
+ Dificultad (short nivel)
+ void configurar (short nivel)
+ short obtenerNivel ( )
NivelRibera
- float scrollX
- float limiteScrollIzq
- float limiteScrollDer
- float anchoMundo
- float posicionOrilla
- vector <Plataforma*> plataformas
+ void cargar ( )
+ void actualizar (float dt)
+ bool verificarVictoria ( )
+ void actualizarScroll ( )
+ void resolverPlataformas ( )
Trampa
# short danio
# bool mortal
# ModeloFisico* modelo
+ Trampa float x, float y, float ancho, float alto)
+ virtual void aplicarEfecto (Personaje& p, float dt)
+ void actualizar (float dt)
+ ~Trampa ( )
GiftFortachon
- float duracionInvulnerable
+ GiftFortachon (float x, float y)
+ void aplicarEfecto (Personaje& p)
GiftSaltarin
- Short saltosExtra
+ GiftSaltarin (float x, float y)
+ void aplicarEfecto (Personaje& p)
Plataforma
- float x
- float y
- float ancho
- float alto
+ Plataforma (float x, float y, float ancho, float alto)
+ bool colisiona (const Entidad& e)
+ float superficieY ( )
Lodazal
- float coeficienteViscosidad
+ Lodazal (float x, float y, float ancho, float alto)
+ void aplicarEfecto (Personaje& p, float dt)
Resbaladero
- float coeficienteFriccion
- float duracionDeslizamiento
+ Resbaladero (float x, float y, float ancho, float alto)
+ void aplicarEfecto (Personaje& p, float dt)
Vibora
- float tiempoAturdido
+ Vibora (float x, float y)
+ void aplicarEfecto (Personaje& p, float dt)
TroncoFlotante
- float velocidadDeriva
+ TroncoFlotante (float x, float y, float ancho, float alto)
+ void aplicarEfecto (Personaje& p, float dt)
+ void actualizar (float dt)
Remolino
- float radioNucleo
- float intensidad
+ Remolino (float x, float y, float radio)
+ void aplicarEfecto (Personaje& p, float dt)
Proyectil
- short danio
- short tipo
- float tiempoVuelo
- float distanciaMinima
- float errorX
- short resultado
- ModeloFisico* modelo
+ Proyectil (float x, float y, float v0, float angulo, short tipo)
+ void actualizar (float dt)
+ void evaluarImpacto (const Personaje& p)
+ bool terminado ( )
+ short obtenerResultado ( )
+ float obtenerDistanciaMinima ( )
+ float obtenerErrorX ( )
+ ~Proyectil ( )
ModeloFisico
# float tiempo
+ virtual void aplicar (Entidad& e, float dt)
+ virtual void reiniciar ( )
MovimientoParabolico
- float x0
- float y0
- float v0
- float angulo
- float gravedad
- short direccion
+ MovimientoParabolico (float x0, float y0, float v0, float angulo)
+ void aplicar (Entidad& e, float dt)
+ float posicionX (float t)
+ float posicionY (float t)
Viscosidad
- float coeficiente
- float velocidadMinima
+ Viscosidad (float k)
+ void aplicar (Entidad& e, float dt)
Friccion
- float mu
- float gravedad
+ Friccion (float mu)
+ void aplicar (Entidad& e, float dt)
+ bool detenido (const Entidad& e)
Vortice
- float centroX
- float centroY
- float circulacion
- float radioNucleo
- float radioInfluencia
+ Vortice (float cx, float cy, float circulacion, float radio)
+ void aplicar (Entidad& e, float dt)
+ bool dentroDelNucleo (const Entidad& e)
Enemigo
# float cadencia
# float tiempoDesdeDisparo
# float rangoAlcance
# float probabilidadGift
# short tipoProyectil
# AgenteLanzador* agente
# vector <Proyectil*> proyectiles
+ Enemigo (float x, float y, const Dificultad& d)
+ virtual void actualizar (float dt, Personaje& p)
+ void lanzar (const Personaje& p)
+ Gift* soltarGift ( )
+ void evaluarProyectiles (Personaje& p, float dt)
+ void limpiarProyectiles ( )
+ ~Enemigo ( )
Mono
- float alturaRama
+ Mono (float x, float y, const Dificultad& d)
+ void actualizar (float dt, Personaje& p)
Cazador
- short orilla
- float distanciaOrilla
+ Cazador (float x, float y, short orilla, const Dificultad& d)
+ void actualizar (float dt, Personaje& p)
AgenteLanzador
- Percepcion* percepcion
- Razonamiento* razonamiento
- Accion* accion
- Aprendizaje* aprendizaje
+ AgenteLanzador (const Dificultad& d)
+ Proyectil* ciclo (const Personaje& p, float xAgente, float yAgente)
+ void aprender (short resultado, float distanciaMinima, float errorX)
+ ~AgenteLanzador ( )
Accion
- bool animado
- float tiempoAnimacion
- short ultimoContexto
- short ultimaEstrategia
+ Proyectil* lanzar (float x, float y, const Razonamiento& r, short tipo)
+ void iniciarAnimacion ( )
+ void actualizar (float dt)
+ short obtenerUltimoContexto ( )
+ short obtenerUltimaEstrategia ( )
Percepcion
- float jugadorX
- float jugadorY
- float jugadorVx
- float jugadorVy
- float distancia
- float rangoPercepcion
- short estadoJugador
- short ultimoResultado
+ void sensar (const Personaje& p, float xAgente, float yAgente)
+ short clasificarEstado (const Personaje& p)
+ bool jugadorEnRango ( )
Razonamiento
- float epsilon
- float gravedad
- short estrategiaElegida
- float velocidadInicial
- float angulo
- Aprendizaje* memoria
+ void decidir (const Percepcion& per)
+ short discretizarContexto (const Percepcion& per)
+ short elegirEstrategia (short contexto)
+ void calcularTiro (const Percepcion& per)
Aprendizaje
- Vector <vector<float>> puntajes
- float factorAnticipacion
- float alfa
- float beta
- short maxHistorial
- deque <Experiencia> historial
+ Aprendizaje (const Dificultad& d)
+ void aprender (short ctx, short est, short resultado, float distMin, float errorX)
+ float calcularRecompensa (short resultado, float distMin)
+ void actualizarPuntaje (short ctx, short est, float recompensa)
+ void ajustarAnticipacion (float errorX)
+ short mejorEstrategia (short ctx)
+ float obtenerFactorAnticipacion ( )
Experiencia
- short contexto
- short estrategia
- short resultado
- float distanciaMinima
- float errorX
- float recompensa
+ Experiencia (short ctx, short est, short res, float dis, float err, float rec)
<Public>
Posee>
Recorre>
Consultas>
1
0...1
0...N
0..N
1...N
0..1